

HiDiTingV100 典型外设适配

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档详细描述了 HiDiTingV100 的典型外设适配开发指导，包括 LCD、TP、Button、Nand Flash、Nor Flash 等典型穿戴产品外设，帮助用户快速进行产品适配。

产品版本

与本文档对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 LCD 适配1

1.1 配置文件介绍 1

1.2 适配步骤..... 1

1.3 常见问题解决方案 5

2 TP 适配6

2.1 驱动概述..... 6

2.2 适配步骤..... 6

2.3 常见问题..... 8

3 按键适配.....9

4 NandFlash 适配12

4.1 驱动适配..... 12

4.2 常见问题..... 15

4.3 功能验证..... 16

5 NorFlash 适配17

5.1 配置文件说明 17

5.1.1 文件说明..... 17

5.1.2 参数介绍..... 17

5.2 适配步骤..... 22

5.2.1 Flash ID 及枚举值配置..... 22

5.2.2 Flash 相关参数配置 23

5.2.3 QE 模式配置 28

5.2.4 部分 Flash 进入 XIP 的特殊配置	29
5.3 常见问题.....	31
6 EMMC 适配.....	32

Draft

插图目录

图 1-1 g_lcd_driver_ops 配置示意图 2

图 1-2 bus_cfg 配置示意图 3

图 1-3 屏幕初始化配置序列 4

图 4-1 Nandflash 存储组织图 13

图 5-1 flash 进 XIP 模式配置示意图..... 20

图 5-2 Flash 参数列表..... 21

图 5-3 QE 模式配置表..... 22

图 5-4 Flash ID 枚举定义 23

图 5-5 Flash 参数列表..... 23

图 5-6 flash suspend 和 resume 接口配置示意图..... 24

图 5-7 需要配置 QE bit 的 flash 列表 29

图 5-8 flash 特性表 30

图 5-9 flash unique id 配置图..... 30

1 LCD 适配

1.1 配置文件介绍

1.2 适配步骤

1.3 常见问题解决方案

1.1 配置文件介绍

LCD 模块支持快速可扩展，用户新屏幕在适配时，需要修改配置文件“lcd_private.c”和“lcd_config.c”。

- lcd_private.c: 该文件提供了屏幕驱动的内部私有接口，主要包含 LCD 的上下电时序控制。不同的 LCD 屏幕，上下电时序也不同，因此需要用户根据 LCD 屏幕手册自行适配。
- lcd_config.c: 该文件提供了所有支持的屏幕型号配置表，包括屏幕分辨率、像素格式及屏端操作指令等信息。适配不同屏幕时，只需要用户修改文件“lcd_config.c”中的配置参数即可，主要包括 DCS 协议规定的通用配置命令，具体可参考下文适配步骤。

1.2 适配步骤

LCD 适配步骤如下：

步骤 1 配置屏幕 IO 管脚：

修改“drivers\drivers\driver\lcd\lcd_private.c”，根据屏幕上电要求配置屏幕的复位（rst）和使能（vci_en）管脚。

步骤 2 增加屏幕配置:

修改 “drivers\drivers\driver\lcd\lcd_config.c”，增加屏幕配置，主要在全局变量 “g_lcd_driver_ops” 中增加新屏幕对应的配置，即一组 lcd_drv_cfg 结构体，如图 1-1 所示。

图1-1 g_lcd_driver_ops 配置示意图

```
const lcd_drv_cfg g_lcd_driver_ops[] = {
{
    .bus_type = BUS_DISPLAY_MIPI,
    .bus_cfg = &g_panel_454x454_60_config,
    .x_start = 0xe,
    .y_start = 0,
    .fps = 60,
    .lcd_id = LCD_EDO_454X454_SCREEN_ID,
    .display_on_cmds = &g_edo_454x454_screen_init_on_cmds,
    .display_off_cmds = &g_display_off_cmds,
    .enter_idle_cmds = &g_idle_in_cmds,
    .exit_idle_cmds = &g_idle_out_cmds,
    .te_on_cmds = &g_te_on_cmds,
    .te_off_cmds = &g_te_off_cmds,
    .set_brightness_cmds = &g_brightness_cmds,
    .sleep_in_cmds = &g_sleep_in_cmds,
    .sleep_out_cmds = &g_sleep_out_cmds,
    .lcd_id_info.reg_addr = 0x04,
    .lcd_id_info.read_len = 3,
},
}
```

各参数说明如下:

- bus_type: 只支持 MIPI 或者 QSPI, 该参数的类型为 lcd_bus_type_t。
- bus_cfg: MIPI 接口主要配置消隐区大小, QSPI 接口主要配置屏幕实际像素大小, 因此需要客户结合屏幕手册对该参数进行配置。修改主要涉及 sync_info 字段, 其他配置字段为 MIPI 协议相关, 无需修改, 具体可参考图 1-2。

注意:

- a. out_mode 只支持 OUT_MODE_DSI_CMD, 不支持 OUT_MODE_DSI_VIDEO。
- b. pixel_clk 需要大于 sync_info 中的 $(hsa+hbp+hact+hfp) * (vsa+vbp+vact+vfp) * 60 / 1000$ (帧率配置 60fps)

- c. $\text{phy_data_rate} = \text{pixel_clk} * 24 / 1000$ ，如果计算出来的值大于 500，需要调整降低帧率，减小 pixel_clk 。
- d. 当修改了 pixel_clk 时，需要同步修改
“middleware\services\gpu\drv\dpu\hal\dpu_hal_crg.c” 中函数
 $\text{dpu_crg_set_vedio_mclk_div_ctl}$ 的 dpu_mclk_div 值，该值的后两位需要与
 pixel_clk 的 16 进制值保持一致，如下 $\text{pixel_clk} = 20000\text{kHz}$ ，换算为 MHz
为 20，对应的 16 进制为 0x14。

```

65: td_void dpu_crg_set_vedio_mclk_div_ctl(td_bool mclk_cken)
66: {
67:     .../*bit[26]:enable, bit[25:13]:div_denom, bit[12:0]:div_num*/
68:     ...volatile u_vedio_crg_mclk_div_ctl mclk_div_ctl;
69:     ...uapi_reg_read(CRG_VEDIO_CRG_MCLK_DIV_CTL, mclk_div_ctl.u32);
70:     ...mclk_div_ctl.bits.dpu_mclk_div = 0x4126014; /*147M * 20/147 = 20M*/

```

图1-2 bus_cfg 配置示意图

```

const combo_dev_cfg_t g_panel_454x454_60_config = {
    .devno = 0,
    .lane_id = {0, -1},
    .out_mode = OUT_MODE_DSI_CMD,
    .out_format = OUT_FORMAT_RGB_24BIT,
    .sync_info = {
        .hsa_pixels = 10, // hsa 10 pixels
        .hbp_pixels = 10, // hbp 10 pixels
        .hact_pixels = 454, // hact 454 pixels
        .hfp_pixels = 10, // hfp 10 pixels
        .vsa_lines = 4, // vsa 4 lines
        .vbp_lines = 12, // vbp 12 lines
        .vact_lines = 454, // vact 454 lines
        .vfp_lines = 10, // vfp 10 lines
    },
    .phy_data_rate = 480, // 480Mbps
    .pixel_clk = 20000, // 20000KHz
};

```

- x_start: 像素水平方向起始存放像素位置。
- y_start: 像素垂直方向起始存放像素位置。
- fps: 可忽略。
- lcd_id: 屏幕 ID 值，初始化时会读取 $\text{lcd_id_info.reg_addr}$ 的值和该配置比较，获取 g_lcd_driver_ops 中的配置进行屏幕配置。

- display_on_cmds: 屏幕初始化配置序列, 参考图 1-3。

图1-3 屏幕初始化配置序列

```
/* init sequence for EDO 454*454 lcd */
const lcd_cmd_seq g_edo_454x454_screen_init_on[] = {
    {1, 2, {0xfe, 0x01}, 0},
    {1, 2, {0x30, 0x43}, 0}, /* set LCD frequency to 15HZ */
    {1, 2, {0xfe, 0x00}, 0},
    {1, 2, {0x3a, 0x77}, 0},
    {1, 2, {0x53, 0x20}, 0},
    {1, 2, {0x51, 0xff}, 0},
    {0, 5, {0x2a, 0x00, 0x0e, 0x01, 0xd3}, 0},
    {0, 5, {0x2b, 0x00, 0x00, 0x01, 0xc5}, 0},
    {1, 3, {0x44, 0x01, 0xc5}, 0},
    {1, 2, {0x35, 0x00}, 0},
    {1, 1, {0x11}, 120},
    {1, 1, {0x29}, 0},
};
```

第一个参数: 1 表示 dcs 命令, 0 表示 gen 命令, 此参数需根据当前命令类型填写。

第二个参数: 命令长度 (命令+参数)。

第三个参数: 命令内容。

第四个参数: 命令发送完成后延迟时间, 单位: ms。

- display_off_cmd ~ sleep_out_cmd 需要对照当前屏幕命令去实现。
- lcd_id_info.reg_addr: 读取屏幕 ID 寄存器地址。
- lcd_id_info.read_len: 屏幕 ID 长度。

步骤 3 适配 GUI 屏幕分辨率, 需要修改如下文件

“drivers/boards/brandy_evb/product/product_evb4_standard.h” 中的 RESOLUTION_WIDTH 和 RESOLUTION_HEIGHT, 默认分支配置为 454×454, 修改为新屏幕大小 (QSPI 等其他规格, 请修改对应分支)。

----结束

以上即完成屏幕适配。

1.3 常见问题解决方案

问题 1：屏幕撕裂问题

解决方案：调整 Tear_Scanline。使屏幕刷新时间晚于 DPU 送显时间，具体操作如下：在屏幕初始化序列中添加 0x44 的命令，参数设置范围为[1, 屏幕高度]，该命令的效果是让屏幕刷新的时间延迟 N 行，具体设置值可根据裂屏时大致行数适配，如下图所示。

```
114: /*.init.sequence.for.EDO.454*454.lcd.*/
115: const lcd_cmd_sequ g_edo_454x454_screen_init_on[] = {
116:     ... {1, 2, {0xfe, 0x01}, 0},
117:     ... {1, 2, {0x30, 0x43}, 0}, /*.set.LCD.frequency.to.15HZ.*/
118:     ... {1, 2, {0xfe, 0x00}, 0},
119:     ... {1, 2, {0x3a, 0x77}, 0},
120:     ... {1, 2, {0x53, 0x20}, 0},
121:     ... {1, 2, {0x51, 0xff}, 0},
122:     ... {0, 5, {0x2a, 0x00, 0x0e, 0x01, 0xd3}, 0},
123:     ... {0, 5, {0x2b, 0x00, 0x00, 0x01, 0xc5}, 0},
124:     ... {1, 3, {0x44, 0x01, 0xc5}, 0},
125:     ... {1, 2, {0x35, 0x00}, 0},
126:     ... {1, 1, {0x11}, 120},
127:     ... {1, 1, {0x29}, 0},
    ... }
```

问题 2：低带宽问题

消隐区配置过小时，容易发生低带宽导致的黑屏，可适当增加消隐区来解决该问题，如下配置为红框值时有低带宽问题，调整为蓝框中值后低带宽问题消失。

低带宽的表现：AT^GPU_SAMPLE=dpu_proc 0 中 “lowband_cnt” 非 0 且一直增加。

	参数列表	参数设置	参数设置	已知屏幕配置	
				GT2	GT5
参数设置	hsa_pixels	10	4	10	20
	hbp_pixels	10	12	10	40
	hact_pixels	390	390	454	466
	hfp_pixels	10	16	10	20
	vsa_lines	4	2	4	4
	vbp_lines	12	6	12	12
	vact_lines	450	450	454	466
	vfp_lines	10	6	10	20
	output format bits	24	24	24	24
	framerate	60	60	60	60
	lane_num	1	1	1	1

2 TP 适配

本章节针对适配新 TP 器件提供指导，包含驱动目录简介、适配步骤、以及常见问题分析。

2.1 驱动概述

2.2 适配步骤

2.3 常见问题

2.1 驱动概述

当前主干版本支持的触控型号有 ztw523、tma525b。

驱动目录：

- drivers/drivers/driver/touch 目录下是触控公共处理相关源文件。
- drivers/drivers/driver/touch/ztw523_touch 目录下是 ztw523 型号触控使用的源文件。
- drivers/drivers/driver/touch/tma525b_touch 目录下是 tma525b 型号触控使用的源文件。

2.2 适配步骤

以 ZTW523 为例，TP 适配步骤如下：

- 步骤 1 触控在项目实际使用中仅编译一种型号的源文件，因此适配新型号的触控时需要新建对应型号的目录：drivers/drivers/driver/touch/xxx_touch。

- 步骤 2 适配触控通信 I2C 的相关配置，如 bus id 和从机地址，参考头文件 “drivers/drivers/driver/touch/ztw523_touch/ztw523_ctrl.h”。

```
#define ZTW523_I2C_BUS          I2C_BUS_1    // bus id
#define ZTW523_I2C_ADDR        0x20         // slave通信地址
```

- 步骤 3 适配触控的中断管脚和复位管脚。

```
#define ZTW523_INT_GPIO        S_AGPIOS      // 中断管脚
#define ZTW523_RESET_GPIO     S_AGPIO4      // 复位管脚
```

- 步骤 4 触控基于 I2C 通信，需要在 “application/brandy/brandy_standard/thread_pri_init.c” 文件中初始化 I2C，例如 ztw523 调用以下接口：

```
ext_errno ztw523_host_init_sample(void)
```

- 步骤 5 新建目录下的源文件可用已有型号的触控源文件为模板修改，用户通过触控厂商获取 demo 或触控手册，优先适配初始化和中断处理部分。

```
ext_errno ztw523_init(void)                                // 触控初始化
ext_errno ztw523_irq_callback(uint8_t *data_buf, uint8_t data_len) // 中断处理，获取报点信息
```

- 步骤 6 适配触控芯片的通信接口，不同芯片的 I2C 交互方式不同，用户需要对比触控手册适配 I2C 通信接口。

```
ext_errno tp_i2c_data_read(uint16_t reg_addr, uint8_t *data_buf, uint32_t data_len);
ext_errno tp_i2c_cmd_write(uint16_t cmd);
ext_errno tp_i2c_reg_write(uint16_t reg_addr, uint16_t reg_cfg);
ext_errno tp_i2c_data_write(uint16_t reg_addr, uint8_t *data_buf, uint32_t data_len);
```

- 步骤 7 适配触控芯片上电，用户需要对比触控手册适配上电时序。

```
ext_errno ztw523_reset(void)
```

- 步骤 8 适配触控公共代码调用的函数指针，完成 “.touch_init” 和 “.touch_get_tpinfo”，即可支持基础报点功能。

```
touch_peripheral_api g_ztw523_api = {
    .touch_init = ztw523_drv_init,
    .touch_deinit = ztw523_drv_deinit,
    .touch_get_tpinfo = ztw523_drv_get_info,
    .touch_suspend = ztw523_drv_suspend,
    .touch_resume = ztw523_drv_resume,
    .touch_sleep = ztw523_drv_sleep,
    .touch_bus_init = ztw523_host_peripheral_init,
```

```
.register_callback = ztw523_register_handle,  
.unregister_callback = ztw523_unregister_handle,  
};
```

步骤 9 完善步骤 8 中的其他接口。

----结束

2.3 常见问题

1. 芯片与触控通信异常 nack，检查触控上电时序、通信使用的 bus id、slave 通信地址是否正确。
2. 触摸屏时中断不上报，检查中断管脚适配是否正确、连接是否正常。
3. 触摸屏时中断可以正常上报，但获取的坐标、事件等信息错误，对比触控手册确保获取方式正确。
4. 触控可以正常报点时，需要确保中断处理时间不能过长，否则影响上层送显效果。

3 按键适配

本章节针对 GPIO 按键扩展提供适配指导。

支持的 GPIO 按键可扩展最多 7 个。

按键适配步骤如下：

- 步骤 1 适配 “application/brandy/brandy_standard/input_app.c” 文件下的 g_brandy_button_pin_map，例如适配 BUTTON_1 为第一个 GPIO 按键，需要将其 PIN_NONE 适配为按键实际使用的 GPIO 管脚。

```
button_map_t g_brandy_button_pin_map[BUTTON_PIN_MAP_MAX] = {  
    { PIN_NONE, BUTTON_PWR },  
    { PIN_NONE, BUTTON_1 },  
    { PIN_NONE, BUTTON_2 },  
    { PIN_NONE, BUTTON_3 },  
    { PIN_NONE, BUTTON_4 },  
    { PIN_NONE, BUTTON_5 },  
    { PIN_NONE, BUTTON_6 },  
    { PIN_NONE, BUTTON_7 },  
};
```

- 步骤 2 低功耗版本需要确保按键对应的 GPIO 管脚打开 IE，在 “drivers/boards/brandy_evb/board_config/board_evb.h” 文件中修改对应管脚的 IE 配置。

```
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,  
  HAL_PIO_IE_DISABLE }, // S_AGPIO_L23  
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,  
  HAL_PIO_IE_DISABLE }, // S_AGPIO_L24
```

例如对应管脚是 S_AGPIO_L23，将该行的 HAL_PIO_IE_DISABLE 改为 HAL_PIO_IE_ENABLE。

步骤 3 在 “application/brandy/brandy_standard/input_app.h” 中适配新增按键事件枚举，默认只有 BUTTON_1 事件。例如要新增两个 GPIO 按键，需要在 BUTTON1 事件枚举之后新增一组 BUTTON2 的事件枚举，以此类推。

```
typedef enum {
    /* power key */
    BUTTON_PWR_START = 0,
    BUTTON_PWR_CLICKUP,
    BUTTON_PWR_CLICKUP_2,
    BUTTON_PWR_LONGPRESS_1S,
    BUTTON_PWR_LONGPRESS_5S,
    BUTTON_PWR_INVALID,

    /* function key */
    BUTTON1_START,
    BUTTON1_CLICKUP,
    BUTTON1_CLICKUP_2,
    BUTTON1_LONGPRESS_1S,
    BUTTON1_LONGPRESS_5S,
    BUTTON1_INVALID,

    /* new key */
    BUTTON2_START,
    BUTTON2_CLICKUP,
    BUTTON2_CLICKUP_2,
    BUTTON2_LONGPRESS_1S,
    BUTTON2_LONGPRESS_5S,
    BUTTON2_INVALID,

    BUTTON_BUTT
} event_button_t;
```

步骤 4 在 “application/brandy/brandy_standard/input_app.c” 文件的 button_event_get 接口中，适配新增按键事件处理，默认只有 BUTTON_1 事件。例如要新增两个 GPIO 按键，需要在 BUTTON1 事件处理之后新增一组 BUTTON2 的事件处理，以此类推。

```
static event_button_t button_event_get(button_id_t id, button_report_state_t state)
{
    event_button_t event = BUTTON_PWR_START;

    if (id == BUTTON_1) {
        event = BUTTON1_START;
    }
}
```

```
/* new key */  
if (id == BUTTON_2) {  
    event = BUTTON2_START;  
}  
.....  
}
```

----结束

说明

若只适配一个 GPIO 按键，完成[步骤 1](#)、[步骤 2](#) 即可，否则需要全部步骤适配完成。

4 NandFlash 适配

须知

NandFlash 当前在正常场景是四线写四线读，异常场景是单线写单线读。

在适配之前请熟读 NandFlash 手册，对 NandFlash 有一定了解，需要对每个指令的波形有较为清晰的了解，需要知道当前厂商的波形与已支持的厂商的波形差异，如果波形有差异，还需要调整 QSPI 层的代码，以下内容以假设具体指令波形与已支持的厂商的波形相同的情况为例。

4.1 驱动适配

4.2 常见问题

4.3 功能验证

4.1 驱动适配

SDK 当前已经适配了大部分厂商的 NandFlash，理论上无需修改驱动代码，但仍然存在小部分厂商的配置与默认配置有一点区别，比如华邦，如果是华邦的 NandFlash，需要将如下代码中 “g_ds35x1g_driver” 默认的驱动改为 “g_w25n01g_driver”。

```
struct nand_driver_struct *get_nand_flash(void)
{
    return &g_ds35x1g_driver;
}
```

其他厂家的适配请参考代码

“drivers/drivers/driver/nand_flash/nand/nandflash_w25n01g.c”，需要在

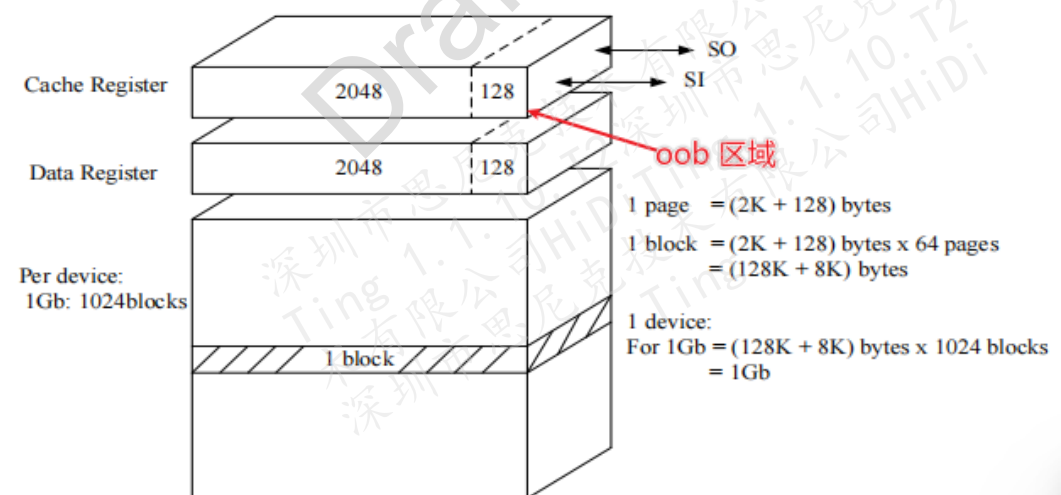
“drivers/drivers/driver/nand_flash” 目录中添加对应的型号。如果需要使用到镜像烧录功能，请将修改同步一份到目录 “bootloader/flashboot/brandy/nandflash”，并同步验证镜像烧录功能是否正常，比对烧录文件是否完整。

主要代码适配点以下面代码为例，不同厂家可能初始化配置不同，请联系 NandFlash 厂家 FAE 确认初始化配置。

```
ret = mc_flash_write_sr(CMD_FLASH_SR1_ADDR, 0x0);
if (get_nand_flash()->enable_ecc == true) {
    ret = mc_flash_write_sr(CMD_FLASH_SR2_ADDR, 0x11);
} else {
    ret = mc_flash_write_sr(CMD_FLASH_SR2_ADDR, 0x01);
}
```

常见 NandFlash 器件差异适配主要是 OOB 区域长度不同，器件的 BLOCK 数量不同，以图 4-1 为例，图中可知该器件 OOB 长度为 128Byte，BLOCK 数量为 1024，适配前需要在手册中查找 OOB 长度信息与 BLOCK 数量信息，获取 OOB 的长度与 BLOCK 的数量。

图4-1 Nandflash 存储组织图



NandFlash 器件 OOB 区域的长度适配步骤如下：

- 步骤 1 代码适配，首先应检查 NandFlash 手册中标识的 OOB 区域的长度，需要针对不同 OOB 区域差异做适配，将 OOB 长度适配为与 NandFlash 手册一致，代码中的 OOB 长度默认是 64Byte，如果手册中 NandFlash 的 OOB 长度是 128Byte，则需要适配。适配方法如下：

1. 需要修改 “bootloader/flashboot/brandy/nandflash/include/nandflash_ds35x1g.h” 文件中的宏 DS35X1G_BYTES_PER_OOB，将其修改成 128。

```
#define DS35X1G_BYTES_PER_OOB 128
```

2. 需要修改

“W:\brandy_602\build\config\target_config\brandy\mk_fs_image\mkyaffs2tool.py” 文件中的 cfg_obb_size，将其修改为 128。

```
cfg_obb_size = 128
```

3. 修改 OOB 区域内容组织格式，使其兼容先贴后烧与先烧后贴。

A 核编译的代码：

drivers/drivers/driver/nand_flash/nand/include/nandflash_ds35x1g.h

ssb 核编译的代码：

bootloader/flashboot/brandy/nandflash/include/nandflash_ds35x1g.h

修改如下几个宏的定义

```
#define DS35X1G_OOB_TOTAL_LEN 1
#define DS35X1G_OOB_PER_SIZE 32
#define DS35X1G_OOB_1_OFFSET 0
#define DS35X1G_OOB_2_OFFSET 0
#define DS35X1G_OOB_3_OFFSET 0
#define DS35X1G_OOB_4_OFFSET 0
```

步骤 2 镜像编译，重新编译 ssb.bin 和 application.bin。

步骤 3 修改验证，验证 BurnTool 烧录是否正常，将 file.bin、ssb.bin、application.bin 镜像通过 BurnTool 工具烧录到主板中，观测文件系统是否可以正常启动，如果可以正常浏览文件，检查预置在文件系统上的文件内容是否完整，若文件完整则适配成功；如果不能正常启动，请检查以上步骤是否正确。

----结束

NandFlash 的 BLOCK 数量适配步骤如下：

步骤 1 代码适配，修改

“drivers/drivers/driver/nand_flash/nand/include/nandflash_ds35x1g.h” 与

“bootloader/flashboot/brandy/nandflash/include/nandflash_ds35x1g.h” 文件中的 **DS35X1G_TOTAL_BLOCKS**，将其修改成与实际大小一致。

```
#define DS35X1G_TOTAL_BLOCKS 1024
```

步骤 2 镜像编译，重新编译 ssb.bin 和 application.bin。

修改验证，验证 BurnTool 烧录是否正常，把 file.bin 和 ssb.bin application.bin 镜像通过 BurnTool 工具烧录到主板中，观测文件系统是否可以正常启动，如果可以正常启动，检查预置在文件系统中的文件是否完整，检查文件系统总大小是否与 NandFlash 手册中标识的器件大小一致。

----结束

参数介绍

默认配置在 “drivers/drivers/driver/nand_flash/nand/include/nandflash_ds35x1g.h” 和 “bootloader/flashboot/brandy/nandflash/include/nandflash_ds35x1g.h” 文件中。

```
#define DS35X1G_BYTES_PER_PAGE 2048 //每一页大小，无需修改

#define DS35X1G_BYTES_PER_OOB 64 //oob大小

#define DS35X1G_PAGES_PER_BLOCK 64 //每一块的页数。无需修改

#define DS35X1G_TOTAL_BLOCKS 1024 //块数，128Mbyte大小的NandFlash正好是1024块，
如果使用更大的NandFlash需要修改这个数值

#define DS35X1G_OOB_TOTAL_LEN 4 //oob中ecc区域的数量

#define DS35X1G_OOB_PER_SIZE 8 //每一块ecc区域的大小，单位：Byte

#define DS35X1G_OOB_1_OFFSET 0 //oob区域第一块数据起始位置

#define DS35X1G_OOB_2_OFFSET 16 //oob区域第二块数据起始位置

#define DS35X1G_OOB_3_OFFSET 32 //oob区域第三块数据起始位置

#define DS35X1G_OOB_4_OFFSET 48 //oob区域第四块数据起始位置
```

4.2 常见问题

须知

由于 NandFlash 器件支持内部 ECC (Error Correcting Code) 特性，该特性在版本上需要使用宏 “ENABLE_ECC” 进行开启或者关闭，当版本上该特性发生切换（开启 -> 关闭或者关闭 -> 开启）时，由于 NandFlash 内部存储结构发生变化，需要对文件系统进行格式化后再使用。

焊接问题：当硬件连接成功时候可以通过 AT 命令 “AT^GETIDS” 获取器件 ID，如果读取到 NandFlash 的 ID 为 0，或者四线模式读出来的数据始终为 0，或 0xCC 等固定的数据，优先考虑硬件焊接原因。

4.3 功能验证

硬件连接成功以后，请使用如下 AT 命令进行测试，查看是否默认支持。如果驱动功能不正常，请根据具体的情况联系 NANDFLASH 厂家 FAE 定位。

```
AT^NANDW={页码} //写一页数据，数据为循环递增0, 1, 2, 3
```

```
AT^NANDR={页码} //读指定页数据
```

```
AT^NANDE={块号} //擦除某一块，擦除后数据为全FF
```

```
AT^NANDM={块号} //标记某一块为坏块
```

```
AT^NANDC={块号} //检查某一块是否为坏块
```

5 NorFlash 适配

本章节针对新款 NorFlash 器件提供适配指导，包含 NorFlash 参数介绍、适配案例以及常见问题分析。使得产品开发人员能够基于 SDK 代码，快速适配新款 NorFlash 器件。当前 SDK 已经支持 10+ 个厂商多款 NorFlash 器件。

📖 说明

本章节部分内容引用 Flash datasheet 《W25Q128JW 1.8V 128M-BIT SERIAL FLASH MEMORY WITH DUAL, QUAD SPI Industrial Plus Grade》，仅供参考适配。

5.1 配置文件说明

5.2 适配步骤

5.3 常见问题

5.1 配置文件说明

5.1.1 文件说明

Flash 相关的配置参数在 “include\driver\flash” 文件夹相关头文件中，含 Flash ID、flash 枚举值、Flash 读写相关命令、Flash 参数列表、进入 QSPI 模式命令、配置 dummy cycle、进入 XIP 模式配置、Flash QE 命令列表等参数。

5.1.2 参数介绍

- Flash ID
- Flash 枚举值：在 flash_support_manufacturer_t 中添加对应的枚举值。
 - 格式：FLASH_XXX0_XXX1
XXX0 表示 Flash 厂商，XXX1 表示 Flash 具体信号。

- 实例：如添加 MXIC MX25R32, FLASH_MXIC_MX25R32

- Flash 大小

```
#define FLASH_SIZE_4MB 0x400000
#define FLASH_SIZE_8MB 0x800000
#define FLASH_SIZE_16MB 0x1000000
#define FLASH_SIZE_32MB 0x2000000
```

- 进入、退出 XIP 模式命令

请参见“5.2.2 Flash 相关参数配置”章节。

- read dummy cycle

请参见“5.2.2 Flash 相关参数配置”章节。

- Flash 读写相关命令

在对应头文件中添加，当前使用的所有 Flash。读写擦等命令相同，无需修改。

读写状态寄存器命令不一样，需要在配置参数中修改。

Red security cmd, suspend/resume cmd 不同，需要在参数列表中直接修改。

- 进入 QSPI 模式命令

结构类型：

```
typedef struct {
    flash_cmd_type_t cmd_type;
    hal_spi_frame_format_t spi_frfr_mode;
    uint8_t cmd_len;
    uint8_t cmd[FLASH_CMD_LEN_MAX];
} flash_cmd_exe_t;
```

其中：

flash_cmd_type_t 定义为：

```
typedef enum flash_cmd_type {
    FLASH_CMD_TYPE_CMD,
    FLASH_CMD_TYPE_PROCESSING,
    FLASH_CMD_TYPE_CHECK,
    FLASH_CMD_TYPE_END
} flash_cmd_type_t;
```

spi_frfr_m_t 定义参考 hal_spi.h。

进入 QSPI 模式，按照“状态寄存器使能→写状态寄存器→判断是否写入→使能 QPI→退出”流程来配置。实例：

```
static flash_cmd_exe_t g_winbond_w25qxx_enter_qspi_mode_cmd[] = {
    /* Send the Write Enable (WREN) command for the volatile status register. */
    { FLASH_CMD_TYPE_CMD,          HAL_SPI_FRAME_FORMAT_STANDARD, 1,
```

```

{ 0x50 }},
    /* Write Status Register-2 to 0x2, enable the QPI write bit. */
    { FLASH_CMD_TYPE_CMD,          HAL_SPI_FRAME_FORMAT_STANDARD, 2,
{ 0x31, 0x02 }},
    /* Wait until the configuration is complete. */
    { FLASH_CMD_TYPE_PROCESSING, HAL_SPI_FRAME_FORMAT_STANDARD, 3,
{FLASH_RDSR1_CMD, 0, 0} },
    /* Enable the QPI mode. */
    { FLASH_CMD_TYPE_CMD,          HAL_SPI_FRAME_FORMAT_STANDARD, 1,
{ 0x38 }},
    /* Set the dummy cycle to 6, with the max. read frequency being 104 MHz. */
    { FLASH_CMD_TYPE_CMD,          HAL_SPI_FRAME_FORMAT_QUAD, 2, { 0xC0,
0x20 }},
    /* The END command will not be executed. */
    { FLASH_CMD_TYPE_END,          HAL_SPI_FRAME_FORMAT_MAX_NUM, 0,
{ 0 }}
};

```

- 配置 dummy cycle

部分 Flash，在不同速率下，需要配置为不同的 dummy cycle，请参考对应 Flash 手册。

- 进入 XIP 模式配置

结构类型：

```

typedef struct flash_qspi_xip_config {
    uint32_t enter_xip_before_wait_cycles_config;
    uint32_t enter_xip_before_inst_l;
    uint32_t enter_xip_before_addr_l;
    uint32_t enter_xip_before_trans_type;
    uint32_t enter_xip_after_wait_cycles_config;
    uint32_t enter_xip_after_inst_l;
    uint32_t enter_xip_after_addr_l;
    uint32_t enter_xip_after_trans_type;
    bool enter_xip_after_enable_32bit_addr;
    bool enter_xip_after_enable_wrap;
} flash_qspi_xip_config_t;

```

主要配置 dummy cycle、指令长度、地址长度、传输模式、是否使用 32 位地址以及是否需要配置 wrap。其中：

- dummy cycle 需要查看对应 Flash 的 datasheet 确认（cmd 模式 dummy cycle 需要+2）。
- 指令长度默认为 8bit。

- 地址长度默认设置为 32bit。
- 传输模式分为：Standard SPI mode（单线命令+四线数据）和 QPI mode（四线数据+四线命令+四线数据），分别对应 HAL_SPI_TRANS_TYPE_INST_S_ADDR_Q 和 HAL_SPI_TRANS_TYPE_INST_Q_ADDR_Q。

实例：

图5-1 flash 进 XIP 模式配置示意图

```

/* Winbond enter xip mode config. */
/* W25Q32/W25Q64/W25Q128 winbond flash config. */
static flash_qspi_xip_config_t g_winbond_w25qxx_config = {
    /* Enter XIP mode before config. */
    WAIT_CYCLES_4,
    HAL_SPI_INST_LEN_8,
    HAL_SPI_ADDR_LEN_32,
    HAL_SPI_TRANS_TYPE_INST_Q_ADDR_Q,
    /* Enter XIP mode after config. */
    WAIT_CYCLES_4,
    HAL_SPI_INST_LEN_0,
    HAL_SPI_ADDR_LEN_32,
    HAL_SPI_TRANS_TYPE_INST_Q_ADDR_Q,
    /* XIP 32bit addr & wrap config. */
    false,
    false,
};

```

- Flash 参数列表

结构类型：

```

typedef struct flash_device_parameter {
    uint32_t manufacturer_id;
    uint32_t flash_size;
    uint8_t enter_xip;
    uint8_t exit_xip;
    uint8_t read_dummy_clk;
    uint8_t security_reg_cmd;
    uint8_t suspend_cmd;
    uint8_t resume_cmd;
    flash_cmd_exe_t *enter_qspi_mode_cmd;
    flash_cmd_exe_t *enter_xip_mode_cmd;
    flash_qspi_xip_config_t *enter_xip_mode_config;
};

```

```
} flash_device_parameter_t;
```

主要配置：Flash ID、Flash size、进入退出 XIP 模式命令、read dummy cycle、read security reg, suspend/resume cmd、进入 QSPI 命令、配置 dummy cycle、进入 XIP 模式配置。

图5-2 Flash 参数列表

```
static const flash_device_parameter_t g_flash_device_parameter[FLASH_MANUFACTURER_MAX] = {
    { FLASH_MANUFACTURER_MXIC_MX25R32,    FLASH_SIZE_4MB,  0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
      g_mx25rxx_enter_qspi_mode_cmd,      NULL,          &g_mx25rxx_config },
    { FLASH_MANUFACTURER_MXIC_MX25R64,    FLASH_SIZE_8MB,  0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
      g_mx25rxx_enter_qspi_mode_cmd,      NULL,          &g_mx25rxx_config },
    { FLASH_MANUFACTURER_MXIC_MX25U64,    FLASH_SIZE_8MB,  0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
      g_mx25u64_enter_qspi_mode_cmd,      NULL,          &g_mx25u64_config },
    { FLASH_MANUFACTURER_MXIC_MX25U128,   FLASH_SIZE_16MB, 0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
      g_mx25u128_enter_qspi_mode_cmd,     NULL,          &g_mx25u128_config },
    { FLASH_MANUFACTURER_WINBOND_W25Q32,   FLASH_SIZE_4MB,  0x20, 0xFF, WAIT_CYCLES_4, 0x05, 0x75, 0x7A,
      g_winbond_w25qxx_enter_qspi_mode_cmd, NULL,          &g_winbond_w25qxx_config },
}
```

- Flash QE 命令列表

部分 Flash 在切 4 线之前，需要先使能 QE bit。设置 QE bit 按照“状态寄存器使能，写 QE bit，等待配置完成，检测 QE bit 是否生效，退出”流程来配置。

实例：

```
static flash_cmd_exe_t g_winbond_w25qxx_set_qe_cmd[] = {
    /* Check the QE bit. */
    { FLASH_CMD_TYPE_CHECK,      HAL_SPI_FRAME_FORMAT_STANDARD, 3,
      { FLASH_RDSR2_CMD, 0x1, 0x2 }},
    /* Send the Write Enable (WREN) command for the volatile status register. */
    { FLASH_CMD_TYPE_CMD,       HAL_SPI_FRAME_FORMAT_STANDARD, 1,
      { FLASH_WREN_CMD }},
    /* Write Status Register-2 to 0x2, enable the QPI write bit. */
    { FLASH_CMD_TYPE_CMD,       HAL_SPI_FRAME_FORMAT_STANDARD, 2,
      { 0x31, 0x02 }},
    /* Wait until the configuration is complete. */
    { FLASH_CMD_TYPE_PROCESSING, HAL_SPI_FRAME_FORMAT_STANDARD, 3,
      { FLASH_RDSR1_CMD, 0, 0 }},
    /* Check the QE bit. */
    { FLASH_CMD_TYPE_CHECK,      HAL_SPI_FRAME_FORMAT_STANDARD, 3,
      { FLASH_RDSR2_CMD, 0x1, 0x2 }},
    /* The END command will not be executed. */
    { FLASH_CMD_TYPE_END,        HAL_SPI_FRAME_FORMAT_MAX_NUM, 0,
      { 0 }},
};
```

配置命令列表数据类型：

```
typedef struct flash_qe_config {
```

```
uint32_t manufacturer_id;
flash_cmd_exe_t *flash_set_qe_cmd;
} flash_qe_config_t;
```

图5-3 QE 模式配置表

```
static const flash_qe_config_t g_flash_need_set_qe[] = {
    {FLASH_MANUFACTURER_WINBOND_W25Q32, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_WINBOND_W25Q64, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_WINBOND_W25Q128, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_WINBOND_W25Q128JW, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_DOSILICON_FM25M64, g_dosilicon_fm25m64_set_qe_cmd},
    {FLASH_MANUFACTURER_DOSILICON_FM25M4AA, g_dosilicon_fm25m64_set_qe_cmd},
    {FLASH_MANUFACTURER_DOSILICON_DS25M4AB, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_ELITE_EN25SX128A, g_winbond_en25sx128a_set_qe_cmd},
    {FLASH_MANUFACTURER_XTX_XT25Q128D, g_xtx_xt25q128d_set_qe_cmd},
};
```

5.2 适配步骤

5.2.1 Flash ID 及枚举值配置

手册：

Operation Code	M7-M0	ID15-ID8	ID7-ID0
9FH	C8	60	16
90H	C8		15
ABH			15

代码：

需要在 flash_common_config.h 中添加该 Flash 的 ID 和枚举定义。

Flash ID 对应的宏及枚举值如图 5-4 红框所示。

图5-4 Flash ID 枚举定义

```

/* Need to add cmd at xip mode, so it can enter sleep and exit sleep mode. */
#define FLASH_MANUFACTURER_GIGADEVICE_GD25WQ64 0x1765C8
/* Need to add cmd at xip mode, so it can enter sleep and exit sleep mode. */
#define FLASH_MANUFACTURER_GIGADEVICE_GD25LE64EX 0x1760C8 /* LX represents E3IG and ESIG. */
/* Need to add cmd at xip mode, so it can enter sleep and exit sleep mode. */
#define FLASH_MANUFACTURER_GIGADEVICE_GD25LX128 0x1860C8 /* LX represents LB and LE. */
#define FLASH_MANUFACTURER_GIGADEVICE_GD25LQ256 0x1960C8
#define FLASH_MANUFACTURER_DOSILICON_FM25M64 0x1743F8
/* Need to add cmd at xip mode, so it can enter sleep and exit sleep mode. */
#define FLASH_MANUFACTURER_GIGADEVICE_GD25LQ32D 0x1660C8
/* Need to add cmd at xip mode, so it can enter sleep and exit sleep mode. */
#define FLASH_MANUFACTURER_DOSILICON_FM25M4AA 0x1842F8
/* Need to add cmd at xip mode, so it can enter sleep and exit sleep mode. */
#define FLASH_MANUFACTURER_PUYA_P25Q32LX 0x166085 /* LX represents L and LC. */
#define FLASH_MANUFACTURER_PUYA_P25Q64SL 0x176085

```

5.2.2 Flash 相关参数配置

在 flash_config_info.h 的 g_flash_device_parameter 表中添加新 Flash 的信息，需要添加 Flash ID、Flash size、enter xip config、exit xip cmd、dummy cycle、suspend cmd、resume cmd（部分 Flash 型号未适配）、切四线配置、进入 XIP 模式配置、配置使用 24 或 32 位地址模式，配置是否使用 wrap 模式等参数。

图5-5 Flash 参数列表

```

static const flash_device_parameter_t g_flash_device_parameter[FLASH_MANUFACTURER_MAX] = {
    { FLASH_MANUFACTURER_MXIC_MX25R32, FLASH_SIZE_4MB, 0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
      g_mx25r32_enter_qspi_mode_cmd, NULL, &g_mx25r32_config },
    { FLASH_MANUFACTURER_MXIC_MX25R64, FLASH_SIZE_8MB, 0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
      g_mx25r64_enter_qspi_mode_cmd, NULL, &g_mx25r64_config },
    { FLASH_MANUFACTURER_MXIC_MX25U64, FLASH_SIZE_8MB, 0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
      g_mx25u64_enter_qspi_mode_cmd, NULL, &g_mx25u64_config },
    { FLASH_MANUFACTURER_MXIC_MX25U128, FLASH_SIZE_16MB, 0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
      g_mx25u128_enter_qspi_mode_cmd, NULL, &g_mx25u128_config },
    { FLASH_MANUFACTURER_WINBOND_W25Q32, FLASH_SIZE_4MB, 0x20, 0xFF, WAIT_CYCLES_4, 0x05, 0x75, 0x7A,
      g_winbond_w25q32_enter_qspi_mode_cmd, NULL, &g_winbond_w25q32_config },
    { FLASH_MANUFACTURER_WINBOND_W25Q64, FLASH_SIZE_8MB, 0x20, 0xFF, WAIT_CYCLES_4, 0x35, 0x75, 0x7A,
      g_winbond_w25q64_enter_qspi_mode_cmd, NULL, &g_winbond_w25q64_config },
    { FLASH_MANUFACTURER_WINBOND_W25Q128, FLASH_SIZE_16MB, 0x20, 0xFF, WAIT_CYCLES_4, 0x05, 0x75, 0x7A,
      g_winbond_w25q128_enter_qspi_mode_cmd, NULL, &g_winbond_w25q128_config },
    { FLASH_MANUFACTURER_WINBOND_W25Q128JW, FLASH_SIZE_16MB, 0x20, 0xFF, WAIT_CYCLES_4, 0x05, 0x75, 0x7A,
      g_winbond_w25q128jw_enter_qspi_mode_cmd, NULL, &g_winbond_w25q128jw_config },
    { FLASH_MANUFACTURER_WINBOND_W25Q256, FLASH_SIZE_32MB, 0x20, 0xFF, WAIT_CYCLES_2, 0x05, 0x75, 0x7A,
      g_winbond_w25q256_enter_qspi_mode_cmd, NULL, &g_winbond_w25q256_config },
}

```

Flash 相关参数配置步骤如下：

- 步骤 1 根据手册填写 Flash ID。
- 步骤 2 根据手册填写 Flash Size。
- 步骤 3 配置进入、退出 XIP 命令（以 GD25LQ32D 和 MX25R6435F 两款 Flash 为例）
 - 手册 (GD25LQ32D):

Quad I/O Fast Read with “Continuous Read Mode”

The Quad I/O Fast Read command can further reduce command overhead through setting the “Continuous Read Mode” bits (M7-0) after the input 3-byte address (A23-A0). If the “Continuous Read Mode” bits (M5-4) = (1, 0), then the next Quad I/O Fast Read command (after CS# is raised and then lowered) does not require the EBH command code. The command sequence is shown in followed Figure12a. If the “Continuous Read Mode” bits (M5-4) do not equal to (1, 0), the next command requires the first EBH command code, thus returning to normal operation. A “Continuous Read Mode” Reset command can be used to reset (M5-4) before issuing normal command.

手册 (MX25R6435F):

- Note:
- 1. Performance enhance mode, if P7≠P3 & P6≠P2 & P5≠P1 & P4≠P0 (Toggling), ex: A5, 5A, 0F, if not using performance enhance recommend to keep 1 or 0 in performance enhance indicator.
 - 2. Reset the performance enhance mode, if P7=P3 or P6=P2 or P5=P1 or P4=P0, ex: AA, 00, FF

代码 (参考下图红框部分):

```
{ FLASH_MANUFACTURER_GIGADEVICE_GD25LQ32D, FLASH_SIZE_4MB, 0x20, 0x0, WAIT_CYCLES_4, 0x05, 0x75, 0x7A, c0043
g_gigadevice_gd25lq32d_enter_qspi_mode_cmd, NULL, &g_gigadevice_gd25lx_config },

{ FLASH_MANUFACTURER_MXIC_MX25R64, FLASH_SIZE_8MB, 0xA5, 0xAA, WAIT_CYCLES_4, 0x2B, 0xB0, 0x30,
g_mx25rxx_enter_qspi_mode_cmd, NULL, &g_mx25rxx_config },
```

步骤 4 配置 Read dummy cycle, 参考 Flash datasheet.

步骤 5 配置 Suspend、resume 模式 (对应图 5-6 红框部分) 。

手册:

Read Security Registers ⁽⁸⁾	48H	A23-A16	A15-A8	A7-A0	dummy	(D7-D0)	
Program/Erase Suspend	75H						
Program/Erase Resume	7AH						

代码:

图5-6 flash suspend 和 resume 接口配置示意图

```
{ FLASH_MANUFACTURER_GIGADEVICE_GD25LQ32D, FLASH_SIZE_4MB, 0x20, 0x0, WAIT_CYCLES_4, 0x05, 0x75, 0x7A, c0043
g_gigadevice_gd25lq32d_enter_qspi_mode_cmd, NULL, &g_gigadevice_gd25lx_config },
```

步骤 6 配置进入 QSPI 模式命令。

按照写使能→写状态寄存器→使能 QE bit→使能 QPI 模式来配置。

手册:

The Write Status Register (WRSR) command allows new values to be written to the Status Register. Before it can be accepted, a Write Enable (WREN) command must previously have been executed. After the Write Enable (WREN) command has been decoded and executed, the device sets the Write Enable Latch (WEL).

Table2. Commands (Standard/Dual/Quad SPI)

Command Name	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	n-Bytes
Write Enable	06H						
Write Disable	04H						
Volatile SR	50H						
Write Enable							
Read Status Register	05H	(S7-S0)					(continuous)
Read Status Register-1	35H	(S15-S8)					(continuous)
Write Status Register	01H	S7-S0	S15-S8				
Enable QPI	38H						

状态寄存器描述如下：

S15	S14	S13	S12	S11	S10	S9	S8
SUS1	CMP	LB3	LB2	LB1	SUS2	QE	SRP1

S7	S6	S5	S4	S3	S2	S1	S0
SRP0	BP4	BP3	BP2	BP1	BP0	WEL	WIP

代码：

```
/* GD25LQ32D gigadevice flash config. */
static flash_cmd_exe_t g_gigadevice_gd25lq32d_enter_qspi_mode_cmd[] = {
    /* Write enable for Volatils status register. */
    { FLASH_CMD_TYPE_CMD, HAL_SPI_FRAME_FORMAT_STANDARD, 1, { 0x50 } },
    /* Write volatile configuration register, config XIP disable. */
    { FLASH_CMD_TYPE_CMD, HAL_SPI_FRAME_FORMAT_STANDARD, 3, { 0x01, 0x00, 0x02 } },
    /* Enable QPI. */
    { FLASH_CMD_TYPE_CMD, HAL_SPI_FRAME_FORMAT_STANDARD, 1, { 0x38 } },
    /* Set dummy 6, maximum read freq 108MHZ. */
    { FLASH_CMD_TYPE_CMD, HAL_SPI_FRAME_FORMAT_QUAD, 2, { 0xC0, 0x20 } },
    /* End cmd, will not execute and need return. */
    { FLASH_CMD_TYPE_END, HAL_SPI_FRAME_FORMAT_MAX_NUM, 0, { 0 } }
};
```

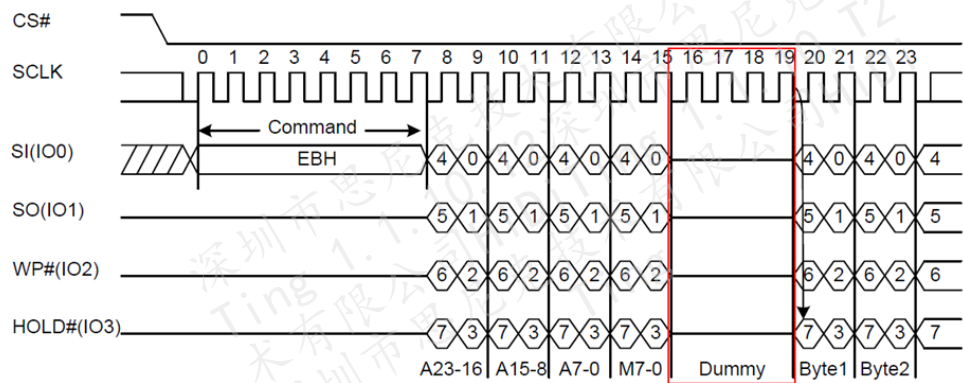
步骤 7 配置 dummy cycle，默认不进行配置，填写 NULL 即可。

步骤 8 配置进入 XIP 模式命令。

需要配置 dummy cycle、指令长度、地址长度、传送模式以及是否使用 wrap。

- 1. dummy cycle、指令长度、地址长度

手册：

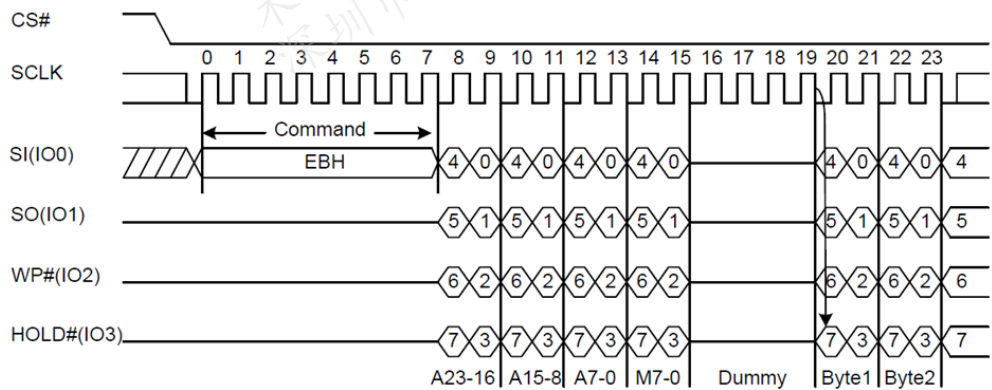


代码:

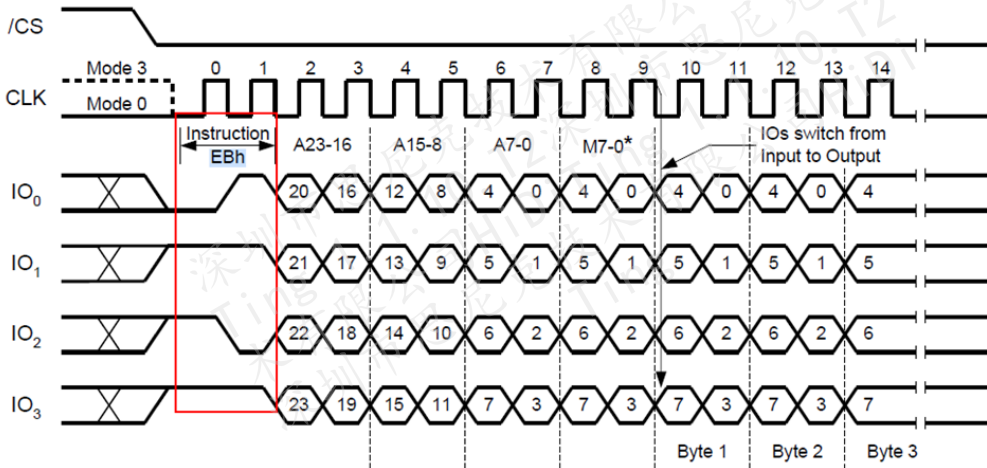
```
/* GD25LX128/GD25LQ32D gigadevice flash config. */
static flash_qspi_xip_config_t g_gigadevice_gd25lx_config = {
    /* The config that before enter XIP mode. */
    WAIT_CYCLES_6,
    HAL_SPI_INST_LEN_8,
    HAL_SPI_ADDR_LEN_24,
    HAL_SPI_TRANS_TYPE_INST_Q_ADDR_Q,
    /* The config that after enter XIP mode. */
    WAIT_CYCLES_6,
    HAL_SPI_INST_LEN_8,
    HAL_SPI_ADDR_LEN_24,
    HAL_SPI_TRANS_TYPE_INST_Q_ADDR_Q,
    /* XIP 32bit addr & wrap config. */
    false,
    false,
};
```

2. 传输模式区别

Standard SPI mode: 单线命令+四线数据



QPI mode: 四线数据+四线命令



代码：

```
/* GD25LX128/GD25LQ32D gigadevice flash config. */
static flash_qspi_xip_config_t g_gigadevice_gd25lx_config = {
    /* The config that before enter XIP mode. */
    WAIT_CYCLES_6,
    HAL_SPI_INST_LEN_8,
    HAL_SPI_ADDR_LEN_24,
    HAL_SPI_TRANS_TYPE_INST_Q_ADDR_Q,
    /* The config that after enter XIP mode. */
    WAIT_CYCLES_6,
    HAL_SPI_INST_LEN_8,
    HAL_SPI_ADDR_LEN_24,
    HAL_SPI_TRANS_TYPE_INST_Q_ADDR_Q,
    /* XIP 32bit addr & wrap config. */
    false,
    false,
};
```

3. xip 32bit mode

手册：

Quad I/O Fast Read	EBH	A23-A0 M7-M0 ⁽⁴⁾	dummy ⁽⁵⁾	(D7-D0) ⁽³⁾			(continuous)
--------------------	-----	-----------------------------	----------------------	------------------------	--	--	--------------

代码：False

手册：

Table2a.1. Commands (QPI) (4-byte mode)

Command Name	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte7
Clock Number	(0,1)	(2,3)	(4,5)	(6,7)	(8,9)	(10,11)	(12,13)
Page Program	02H	A31-A24	A23-A16	A15-A8	A7-A0	(D7-D0)	Next byte
Sector Erase	20H	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase(32K)	52H	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase(64K)	D8H	A31-A24	A23-A16	A15-A8	A7-A0		
Fast Read	0BH	A31-A24	A23-A16	A15-A8	A7-A0	dummy	(D7-D0)
Burst Read with Wrap	0CH	A31-A24	A23-A16	A15-A8	A7-A0	dummy	(D7-D0)
Quad I/O Fast Read	EBH	A31-A24	A23-A16	A15-A8	A7-A0	M7-M0	(D7-D0)

代码： True

4. wrap 模式
- 默认配置为 false。

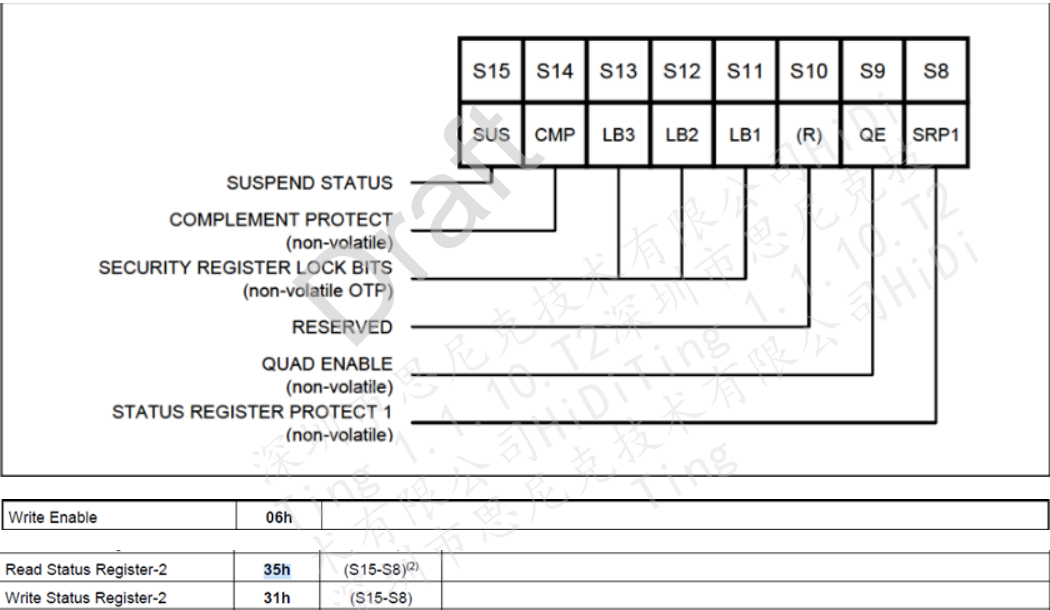
----结束

5.2.3 QE 模式配置

部分 Flash 在切四线前，需要配置 QE 模式，以 winbond 为例。

QE 模式说明：

The Quad Enable (QE) bit is a non-volatile read/write bit in the status register (S9) that allows Quad SPI and QPI operation. When the QE bit is set to a 0 state ((factory default for part numbers with ordering options "IG")), the /WP pin and /HOLD are enabled. When the QE bit is set to a 1(factory default for Quad Enabled part numbers with ordering option "IQ"), the Quad IO2 and IO3 pins are enabled, and /WP and /HOLD functions are disabled.



代码： 写 Flash 状态寄存器 2 里面的 qe bit，然后再回读，等待 qe bit 生效。

图5-7 需要配置 QE bit 的 flash 列表

```
static const flash_qe_config_t g_flash_need_set_qe[] = {
    {FLASH_MANUFACTURER_WINBOND_W25Q32, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_WINBOND_W25Q64, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_WINBOND_W25Q128, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_WINBOND_W25Q128JW, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_DOSILICON_FM25M64, g_dosilicon_fm25m64_set_qe_cmd},
    {FLASH_MANUFACTURER_DOSILICON_FM25M4AA, g_dosilicon_fm25m64_set_qe_cmd},
    {FLASH_MANUFACTURER_DOSILICON_DS25M4AB, g_winbond_w25qxx_set_qe_cmd},
    {FLASH_MANUFACTURER_ELITE_EN25SX128A, g_winbond_en25sx128a_set_qe_cmd},
    {FLASH_MANUFACTURER_XTX_XT25Q128D, g_xtx_xt25q128d_set_qe_cmd},
};
```

```
static flash_cmd_exe_t g_xtx_xt25q128d_set_qe_cmd[] = {
    /* Check QE bit. */
    { FLASH_CMD_TYPE_CHECK, HAL_SPI_FRAME_FORMAT_STANDARD, 3, { FLASH_RDSR2_CMD, 0x1, 0x2 } },
    /* Write enable. */
    { FLASH_CMD_TYPE_CMD, HAL_SPI_FRAME_FORMAT_STANDARD, 1, { FLASH_WREN_CMD } },
    /* Write status register-2 to 0x2, set QE bit. */
    { FLASH_CMD_TYPE_CMD, HAL_SPI_FRAME_FORMAT_STANDARD, 2, { 0x31, 0x02 } },
    /* Wait config done. */
    { FLASH_CMD_TYPE_PROCESSING, HAL_SPI_FRAME_FORMAT_STANDARD, 3, { FLASH_RDSR1_CMD, 0, 0 } },
    /* Check QE bit. */
    { FLASH_CMD_TYPE_PROCESSING, HAL_SPI_FRAME_FORMAT_STANDARD, 3, { FLASH_RDSR2_CMD, 0x1, 0x2 } },
    /* End cmd, will not execute and need return. */
    { FLASH_CMD_TYPE_END, HAL_SPI_FRAME_FORMAT_MAX_NUM, 0, { 0 } }
};
```

5.2.4 部分 Flash 进入 XIP 的特殊配置

某些 Flash 器件需要进行特殊适配，否则无法进入 XIP 模式。

当在调试 Flash 器件中出现进入 XIP 失败的问题时，请先确定 Flash 所支持的能力，从之前的 flash 分析来看，当不支持如图 5-8 所示的 QPI mode 特性时，需要对器件进入 XIP 的 CMD 进行配置。

图5-8 flash 特性表

2. FEATURES

- **New Family of SpiFlash Memories**
 - W25Q32FW: 32M-bit / 4M-byte
 - Standard SPI: CLK, /CS, DI, DO, /WP, /Hold
 - Dual SPI: CLK, /CS, IO₀, IO₁, /WP, /Hold
 - Quad SPI: CLK, /CS, IO₀, IO₁, IO₂, IO₃
 - QPI: CLK, /CS, IO₀, IO₁, IO₂, IO₃
 - Software & Hardware Reset
- **Highest Performance Serial Flash**
 - 104MHz Single, Dual/Quad SPI clocks
 - 208/416MHz equivalent Dual/Quad SPI
 - 50MB/S continuous data transfer rate
 - Min 100K Program-Erase cycles per sector
 - More than 20-year data retention
- **Efficient QPI Mode**
 - As few as 8 clocks to address memory
 - Quad Peripheral Interface (QPI) reduces instruction overhead
 - Allows true XIP (execute in place) operation
 - Outperforms X16 Parallel Flash
- **Low Power, Wide Temperature Range**
 - Single 1.65 to 1.95V supply
 - 4mA active current, <1µA Power-down (typ.)
 - -40°C to +85°C operating range
- **Flexible Architecture with 4KB sectors**
 - Uniform Sector/Block Erase (4K/32K/64K-Byte)
 - Program 1 to 256 byte per programmable page
 - Erase/Program Suspend & Resume
- **Advanced Security Features**
 - Software and Hardware Write-Protect
 - Power Supply Lock-Down and OTP protection
 - Top/Bottom, Complement array protection
 - Individual Block/Sector array protection
 - 64-Bit Unique ID for each device
 - Discoverable Parameters (SFDP) Register
 - 3X256-Bytes Security Registers with OTP locks
 - Volatile & Non-volatile Status Register Bits
- **Space Efficient Packaging**
 - 8-pin SOIC/VSOP 208-mil, XSON 4x4-mm
 - 8-pad WSON 6x5-mm / 8x6-mm
 - 16-pin SOIC 300-mil (additional /RESET)
 - 24-ball TFBGA 8x6-mm (additional /RESET)
 - 12-ball WLCSP
 - Contact Winbond for KGD and other options

当前共发现三款 Flash 需要特殊配置，如图 5-9 所示，我们通过读取 Flash 的 unique ID 来判断，进行添加配置。

```
#define EFLASH_W25Q64JWB YIQ_UNIQUE_ID 0xEC
#define EFLASH_W25Q32JWB YIQ_UNIQUE_ID 0xE9
#define EFLASH_W25Q128JWB YIQ_UNIQUE_ID 0xEB
```

需要在 flash_config_info.h 文件中添加图 5-9 所示三步即可。

图5-9 flash unique id 配置图

```
#define FLASH_W25Q64JWB YIQ_UNIQUE_ID 0xEC
#define FLASH_W25Q32JWB YIQ_UNIQUE_ID 0xE9
#define FLASH_W25Q128JWB YIQ_UNIQUE_ID 0xEB

static const uint32_t g_flash_need_cmd_unique_id[] = {
    FLASH_W25Q32JWB YIQ_UNIQUE_ID,
    FLASH_W25Q64JWB YIQ_UNIQUE_ID,
    FLASH_W25Q128JWB YIQ_UNIQUE_ID };

static const uint32_t g_winbond_flash_manufact_id[] = {
    FLASH_MANUFACTURER_WINBOND_W25Q32,
    FLASH_MANUFACTURER_WINBOND_W25Q64,
    FLASH_MANUFACTURER_WINBOND_W25Q128 };
```

其中第一步所示的 unique ID 可以在启动过程的打印获取到。

综上，某些 Flash 可能需要上述的配置，在调试中如果出现进入 XIP 模式失败问题，需要进行上述特殊配置。

5.3 常见问题

- 焊接问题：
 - 读出来的 Flash ID 为 0。
 - 不管怎么调整 dummy cycle，四线模式读出来的数据始终为 0，或者 0xCC 等固定的数据。
 - 低频时，XIP 可以正常运行，高频时异常。
 - 系统启动时读取 manufactory ID，但是进入 XIP 失败，错误值为 0xFFFFFFFF。 (此问题在某款 NorFlash 上出现，也可能与配置有关，请参见“[5.2.4 部分 Flash 进入 XIP 的特殊配置](#)”章节内容)
- 模式配置错误
 - 日志中出现 timeout 时，要查看是否 QSPI 模式和 QPI 模式配置错误。
 - 切四线前，确认该款 Flash 是否需要配置 QE 模式。
- dummy cycle 配置错误：四线模式，读出的数据有错位。
- resume、suspend 模式：不同厂家，Flash 的状态寄存器以及相应的读写命令不同，如果使用了 resume/suspend 模式，需要对这块接口重新适配。

6 EMMC 适配

由于 EMMC 的初始化流程是按照标准协议 Embedded Multi-Media Card(eMMC) Electrical Standard (5.1)完成的，只要对端的器件支持标准协议，都无需进行特殊适配的。对于初始化流程请参考标准文档《JEDEC STANDARD -JESD84-B51》，本节只对开发过程中的问题和芯片的特殊之处进行说明。

- 本芯片的 EMMC host 支持的最高速率模式为 HS200，并且时钟频率最大支持 100MHz。
- EMMC 的管脚默认空闲态为高，因此除 CLOCK 管脚外的 CMD 和 DATA 管脚都需要默认为高，建议增加上拉电阻或者配置管脚 PAD 为上拉状态（具体配置请参见《HiDiTingV100 低功耗软件 开发指南》的“7.4.1 IO 通用配置”小节）。
- 本驱动中保证了 EMMC 基本的枚举、读、写以及擦的流程，保证上层文件系统的正常运行。
- 芯片进入深睡后，会对 EMMC 的 host 进行下电操作；唤醒后，会对寄存器进行恢复。只要保证深睡时，EMMC 的器件不会下电，就无需重新枚举操作，代码中已对此部分流程进行处理，开发者正常使用即可。
- 在枚举前，请确保 EMMC 器件的供电正常，对应的 RESET 管脚配置正常，由于不同器件的差异，请参考对应的 EMMC 器件手册进行检查。
- 本驱动代码上层基于 liteos 中的开源 mmc 代码实现，底层适配芯片的配置完成。如果需要进行二次开发的，请在开源代码中进行处理。